BỘ KHOA HỌC VÀ CÔNG NGHỆ

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG



BÁO CÁO MÔN HỌC

PHÁT TRIỂN HỆ THỐNG THÔNG MINH

Giảng viên:	**Trần Đình Quế**
Sinh viên:	**Nguyễn Tuấn Tài**
Mã sinh viên:	**B23DCCN731**
Nhóm lớp:	**06**
Nhóm tiểu luận:	**15**

Hà Nội – 2026

Mục lục

1: BÁO CÁO NGHIÊN CỨU HIỂU BIẾT CHUYÊN SÂU VỀ AI AGENT 3
1.1 Khái Niệm Bản Chất và Bước Chuyển Dịch Mô Thức 3
1.2 Phân Loại Các Kiến Trúc Tác Tử Thông Minh 4
1. Tác tử Phản xạ Đơn giản (Simple Reflex Agent) 4
2. Tác tử Dựa trên Mô hình (Model-based Reflex Agent) 4
3. Tác tử Dựa trên Mục tiêu (Goal-based Agent) 4
4. Tác tử Dựa trên Độ Thỏa dụng (Utility-based Agent) 5
5. Tác tử Tự học Thích nghi (Learning Agent) 5
1.3 Các Mô Thức Suy Luận Hiện Đại: ReAct và ReWOO 5
1. Mô thức ReAct 5
2. Mô thức ReWOO 6
1.4 Tác Động Kinh Tế và Ứng Dụng Thực Tiễn Doanh Nghiệp 6
1.5 Thách Thức, Rủi Ro và Biện Pháp Quản Trị Hệ Thống An Toàn 6
2: THỰC NGHIỆM CODE TRÊN NOTEBOOK 8
2.1 Môi Trường Tính Toán và Thiết Lập Ban Đầu 8
2.2 Thực Nghiệm Tác Tử Phản Xạ Đơn Giản - Simple Reflex Agent 9
2.3 Thực Nghiệm Tác Tử Dựa Trên Mô Hình Nội Tại - Model-based Agent 10
2.4 Thực Nghiệm Tác Tử Dựa Trên Mục Tiêu - Goal-based Pathfinder Agent 11
2.5 Thực Nghiệm Tác Tử Dựa Trên Hàm Thỏa Dụng - Utility-based Agent 12
2.6 Thực Nghiệm Tác Tử Tự Học Thích Nghi Q-Learning - Learning 13
2.7 Thực Nghiệm Mô Thức Suy Luận Hiện Đại ReAct Agent 13
2.8 Thực Nghiệm Hệ Đa Tác Tử Cộng Tác - Multi-Agent Collaborative System 15
TÀI LIỆU THAM KHẢO 16

1: BÁO CÁO NGHIÊN CỨU HIỂU BIẾT CHUYÊN SÂU VỀ AI AGENT

1.1 Khái Niệm Bản Chất và Bước Chuyển Dịch Mô Thức

Trong lịch sử phát triển của Trí tuệ Nhân tạo, nhân loại đã chứng kiến nhiều bước nhảy vọt: từ các hệ thống chuyên gia dựa trên luật logic hình thức (Symbolic AI), sự trỗi dậy của máy học thống kê (Machine Learning), đến cuộc cách mạng của mô hình ngôn ngữ lớn (Generative AI). Tuy nhiên, các mô hình ngôn ngữ lớn truyền thống (như GPT-3.5, ChatGPT ban đầu) về bản chất vẫn mang tính chất 'thụ động' (Passive): chúng chỉ trả lời khi được hỏi, hoạt động theo chu kỳ Nhận Prompt -> Sinh Văn bản, và bị cô lập hoàn toàn với môi trường thực thi bên ngoài.

Theo tài liệu nghiên cứu chuyên sâu của Google Cloud [1], IBM [2] và McKinsey & Company [3], sự xuất hiện của Tác tử Nhân tạo (AI Agent / Agentic AI) đánh dấu một bước chuyển dịch mô thức mang tính cách mạng: dịch chuyển từ 'Trợ lý đàm thoại thụ động' (Conversational Assistant) sang 'Thực thể hành động tự chủ' (Autonomous Actor). Một AI Agent là một hệ thống phần mềm thông minh có khả năng cảm nhận môi trường xung quanh, tự chủ lập luận, thiết lập kế hoạch hành động đa bước, gọi các công cụ ngoại vi và tương tác lặp đi lặp lại để hoàn thành một mục tiêu xác định mà không cần con người phải can thiệp cầm tay chỉ việc.

Về mặt khoa học lý thuyết (Russell & Norvig, 2020), một Agent hoàn chỉnh được mô hình hóa theo cấu trúc hình thức:

$$Agent = Architecture + Program \quad \textbf{(1.1)}$$

Và được định nghĩa chặt chẽ thông qua mô hình PEAS:

- Performance Measure (P - Thước đo hiệu năng): Tiêu chuẩn định lượng đo lường mức độ thành công của tác tử (ví dụ: tối ưu chi phí, tỷ lệ hoàn thành nhiệm vụ, độ an toàn y tế).
- Environment (E - Môi trường hoạt động): Miền không gian mà tác tử tương tác (Cơ sở dữ liệu bệnh án, thị trường chứng khoán, hệ điều hành máy tính, mạng Internet).
- Actuators (A - Bộ tác động): Các phương tiện giúp tác tử thực hiện hành động làm thay đổi môi trường (gọi hàm API, gửi email, thực thi mã code, đặt lệnh mua cổ phiếu).
- Sensors (S - Cảm biến / Bộ thu nhận tri giác): Các cơ chế giúp tác tử quan sát và thu thập dữ liệu từ môi trường (HTTP request, log hệ thống, truy vấn cơ sở dữ liệu, camera).

Chu trình vận hành cốt lõi của một AI Agent là vòng lặp tự chủ khép kín (Autonomous Loop) gồm 4 pha tương hỗ:

Perceive \longrightarrow Reason / Plan \longrightarrow Act (Tool Use) \longrightarrow Learn / Reflect **(1.2)**

Đặc tính So sánh	Chatbot AI Truyền thống	Tự động hóa RPA	AI Agent (Agentic AI)
Tính chất hoạt động	Bị động (Chỉ phản hồi khi có prompt)	Tất định (Thực thi theo quy tắc cứng)	Chủ động & Thích ứng (Goal-oriented)
Khả năng lập kế hoạch	Không (Sinh văn bản trực tiếp 1 lần)	Không (Lập trình sẵn theo kịch bản)	Xuất sắc (Tự phân rã công việc đa bước)
Sử dụng công cụ (Tools)	Rất hạn chế hoặc không có	Thao tác trên giao diện UI cố định	Linh hoạt gọi API, Code, Vector DB, Shell
Xử lý ngoại lệ / Lỗi	Sinh ảo giác hoặc xin lỗi người dùng	Dừng chương trình và báo lỗi	Tự phản tư (Self-reflection) và thử lại
Mức độ tự chủ (Autonomy)	Thấp (Phụ thuộc hoàn toàn vào User)	Thấp đến trung bình (Quy trình cứng)	Cao (Tự ra quyết định đạt mục tiêu)
Mô thức phối hợp	Độc lập một-một	Hệ thống đơn luồng	Hệ đa tác tử cộng tác (Multi-Agent)

Bảng 1: Ma trận So sánh Toàn diện Giữa AI Agent, Chatbot Truyền thống và RPA

1.2 Phân Loại Các Kiến Trúc Tác Tử Thông Minh

Trong tài liệu phân tích kỹ thuật của IBM [2] và giáo trình chuẩn mực thế giới của Stuart Russell và Peter Norvig, các tác tử thông minh được phân cấp thành 5 thế hệ kiến trúc với năng lực tư duy tăng tiến:

1. Tác tử Phản xạ Đơn giản (Simple Reflex Agent)

Hoạt động dựa trên tập luật phản xạ Điều kiện - Hành động (Condition-Action Rules: IF condition THEN action). Tác tử chỉ quan sát trạng thái hiện tại và đưa ra hành động tức thời, hoàn toàn không có bộ nhớ quá khứ. Hạn chế cốt tử: chỉ hoạt động hiệu quả khi môi trường có thể quan sát toàn phần (Fully Observable). Nếu môi trường bị che khuất một phần, tác tử dễ rơi vào vòng lặp vô tận.

2. Tác tử Dựa trên Mô hình (Model-based Reflex Agent)

Khắc phục hạn chế của tác tử phản xạ đơn giản bằng cách duy trì một 'Trạng thái nội tại' (Internal State). Trạng thái này lưu trữ mô hình về cách thế giới vận hành (World Model): thế giới biến đổi độc lập ra sao và hành động của tác tử tác động lên môi trường như thế nào. Nhờ đó, tác tử có thể xử lý tốt các môi trường quan sát một phần (Partially Observable Environments).

3. Tác tử Dựa trên Mục tiêu (Goal-based Agent)

Không chỉ dừng lại ở việc biết trạng thái thế giới hiện tại, tác tử được cung cấp một hoặc nhiều Mục tiêu cần đạt tới (Goals). Tác tử áp dụng các thuật toán Tìm kiếm (Search) và

Hoạch định (Planning) để cân nhắc các chuỗi hành động khả dĩ khác nhau nhằm chọn ra lộ trình tối ưu dẫn đến trạng thái mục tiêu.

4. Tác tử Dựa trên Độ Thỏa dụng (Utility-based Agent)

Trong thế giới thực, có nhiều đường đi khác nhau đều có thể đạt được mục tiêu, nhưng với chất lượng, chi phí, thời gian và rủi ro khác nhau. Tác tử dựa trên độ thỏa dụng sử dụng một Hàm thỏa dụng (Utility Function) để ánh xạ một trạng thái sang một giá trị số thực định lượng mức độ hài lòng:

$$U: S \longrightarrow \mathbb{R} \quad \textbf{(1.3)}$$

Tác tử sẽ lựa chọn hành động tối đa hóa độ thỏa dụng kỳ vọng (Maximum Expected Utility - MEU) trong các tình huống đánh đổi phức tạp (Trade-off) giữa tốc độ, độ an toàn và tài nguyên tiêu hao.

5. Tác tử Tự học Thích nghi (Learning Agent)

Đại diện cho đỉnh cao của trí tuệu nhân tạo, gồm 4 thành tố chức năng tương hỗ:

- Thành tố Học hỏi (Learning Element): Chịu trách nhiệm thực hiện các cải tiến và cập nhật tri thức dựa trên kinh nghiệm quá khứ.
- Thành tố Thực thi (Performance Element): Lựa chọn các hành động đối ngoại dựa trên các tri giác tiếp nhận.
- Thành tố Phê bình (Critic): Đánh giá chất lượng hành vi của tác tử dựa trên thang đo hiệu năng bên ngoài, cung cấp tín hiệu phản hồi (Reward / Penalty).
- Thành tố Phát sinh Vấn đề (Problem Generator): Gợi ý các hành động mang tính thử nghiệm, khám phá (Exploration) các chiến lược mới thay vì chỉ khai thác (Exploitation) thói quen cũ.

Cơ chế của Learning Agent kết nối trực tiếp với Học tăng cường (Reinforcement Learning từ tài liệu [4] GeeksforGeeks), nơi tác tử học chính sách hành động tối ưu thông qua phần thưởng môi trường và cập nhật phương trình Bellman.

1.3 Các Mô Thức Suy Luận Hiện Đại: ReAct và ReWOO

Khi tích hợp Mô hình Ngôn ngữ Lớn (LLM) làm bộ não trung tâm của Agent, các kỹ sư phần mềm phải đối mặt với hai thách thức: (1) LLM thuần túy không thể hành động trên thế giới thực, và (2) Việc thực thi mù quáng không suy nghĩ sẽ dẫn đến các hành động sai lầm nghiêm trọng. IBM [2] nhấn mạnh hai mô thức suy luận mang tính đột phá:

1. Mô thức ReAct

ReAct là một phương pháp kết hợp sức mạnh của chuỗi tư duy (Chain-of-Thought) và khả năng tương tác với các công cụ bên ngoài theo một vòng lặp tuần tự chặt chẽ:

- Thought (Suy nghĩ): Tác tử tự phân tích tình huống, lập luận xem mình đang ở bước nào và cần thông tin gì tiếp theo.
- Action (Hành động): Tác tử chọn một công cụ phù hợp từ danh mục Tools và phát lệnh gọi (ví dụ: Search[tên_thuốc] hoặc Calculator[biểu_thức]).
- Observation (Quan sát): Tác tử nhận kết quả trả về từ môi trường/công cụ và đưa kết quả này vào cửa sổ ngữ cảnh.

- Vòng lặp tiếp diễn: Chu trình Thought -> Action -> Observation được lặp lại cho đến khi tác tử có đủ thông tin để sinh ra Final Answer (Câu trả lời cuối cùng).

2. Mô thức ReWOO

Dù ReAct rất mạnh mẽ, việc gọi LLM ở mỗi bước sau mỗi Observation dẫn đến độ trễ lớn và tiêu tốn chi phí token rất cao. Để giải quyết vấn đề này, IBM [2] giới thiệu mô thức ReWOO: Tách rời hoàn toàn pha Lập kế hoạch (Planning Phase) và pha Thực thi Công cụ (Execution Phase). Tác tử lập một đồ thị phụ thuộc kế hoạch hoàn chỉnh (Plan Graph) với các biến giữ chỗ (Placeholders), sau đó một Worker Engine sẽ chạy tuần tự toàn bộ các công cụ và ghép kết quả vào lời nhắc cuối cùng cho LLM tổng hợp, giúp giảm tới 64% chi phí token và tăng tốc độ xử lý gấp nhiều lần.

1.4 Tác Động Kinh Tế và Ứng Dụng Thực Tiễn Doanh Nghiệp

Báo cáo chuyên sâu của McKinsey & Company (QuantumBlack, AI by McKinsey) [3] dự báo Agentic AI sẽ tạo ra một cuộc tái cấu trúc toàn diện chuỗi giá trị toàn cầu, giải phóng hàng nghìn tỷ USD giá trị kinh tế thặng dư. Các tác động nổi bật bao gồm:

- 1. Tự động hóa quy trình nghiệp vụ đầu-cuối (End-to-End Workflow Automation): Thay vì chỉ hỗ trợ từng tác vụ nhỏ lẻ (như viết email hay tóm tắt văn bản), AI Agent có thể quản lý trọn vẹn toàn bộ một quy trình kinh doanh. Ví dụ trong chuỗi cung ứng, hệ thống tác tử có thể tự phát hiện nguy cơ thiếu hụt hàng hóa, liên hệ báo giá với các nhà cung cấp, đàm phán hợp đồng sơ bộ, lập hóa đơn và cập nhật tồn kho ERP hoàn toàn tự động.
- 2. Chuyển dịch mô hình tổ chức sang Đội ngũ Người - AI (Human-AI Hybrid Teams): Các doanh nghiệp hàng đầu đang chuyển dịch từ việc xem AI như công cụ phần mềm tĩnh sang việc quản lý AI Agent như 'Nhân sự số' (Digital Workforce). Con người giữ vai trò lãnh đạo, đặt ra mục tiêu chiến lược và các ranh giới kiểm duyệt (Guardrails), trong khi các đội tác tử chuyên môn hóa tự chủ triển khai chi tiết.
- 3. 6 Phân nhóm Tác tử Doanh nghiệp Theo Google Cloud [1]:
- • Customer Agents: Chăm sóc khách hàng siêu cá nhân hóa, tự xử lý hoàn tiền, đổi trả và tư vấn giải pháp 24/7.
- • Employee Agents: Trợ lý nội bộ hỗ trợ giải đáp chính sách nhân sự, CNTT, đào tạo và giải quyết thủ tục hành chính.
- • Creative Agents: Tự động lên ý tưởng chiến dịch marketing, sản xuất hình ảnh, video và tối ưu hóa nội dung đa kênh.
- • Data Agents: Tự động phân tích dữ liệu lớn, phát hiện dị thường kinh doanh và trích xuất báo cáo thông minh.
- • Code Agents: Tự động sinh mã nguồn, rà soát lỗ hổng bảo mật, viết kiểm thử đơn vị và triển khai CI/CD.
- • Security Agents: Giám sát an ninh mạng thời gian thực, chủ động phát hiện và cách ly các cuộc tấn công mạng.

1.5 Thách Thức, Rủi Ro và Biện Pháp Quản Trị Hệ Thống An Toàn

Cả IBM [2] và McKinsey [3] đều cảnh báo rằng quyền tự chủ càng cao thì rủi ro tiềm ẩn càng lớn. Các thách thức cốt tử gồm:

- 1. Vòng lặp phản hồi vô tận (Infinite Feedback Loops): Khi hai hoặc nhiều tác tử gửi thông điệp qua lại nhưng không có điều kiện dừng rõ ràng, dẫn đến bùng nổ chi phí tính toán và làm treo hệ thống.
- 2. Phụ thuộc dây chuyền đa tác tử (Multi-agent Cascade Failures): Một sai sót nhỏ của tác tử ở tầng đầu vào có thể bị khuếch đại qua các tầng tác tử trung gian, dẫn đến quyết định sai lầm thảm họa ở đầu ra.
- 3. Bảo mật và Rò rỉ dữ liệu (Data Privacy & Prompt Injection): Nguy cơ tác tử bị tấn công chèn mã độc (Indirect Prompt Injection) qua các công cụ ngoại vi, khiến tác tử vô tình tiết lộ dữ liệu nhạy cảm hoặc thực hiện các hành động phá hoại hệ thống.

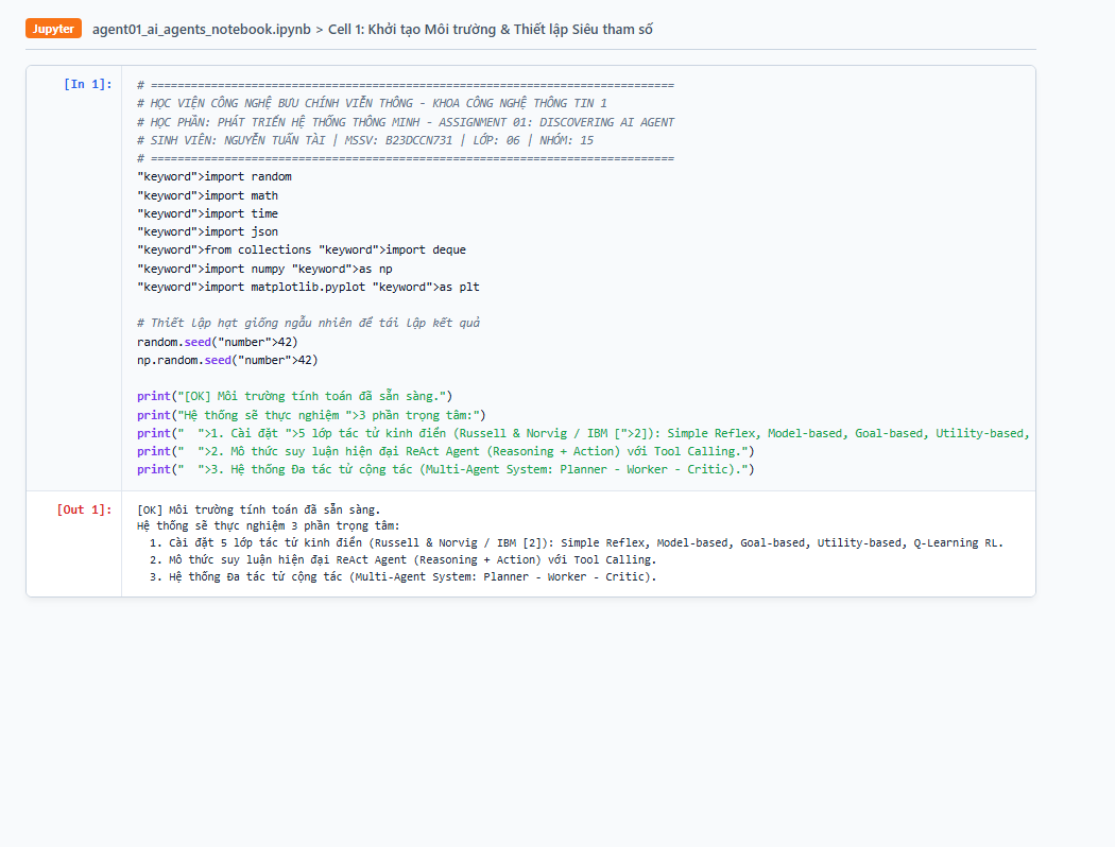
Để kiểm soát các rủi ro trên, IBM [2] đề xuất 4 nguyên tắc thực hành quản trị tối ưu (Best Practices):

- Activity Logs (Nhật ký hành động toàn diện): Ghi lại minh bạch toàn bộ chuỗi Thought, Action, Tool Call và Observation để phục vụ kiểm toán hồi quy.
- Interruption Mechanisms (Cơ chế ngắt khẩn cấp): Thiết lập 'nút dừng khẩn cấp' (Kill-switch) cho phép can thiệp ngắt tác tử ngay lập tức khi phát hiện hành vi vượt khỏi giới hạn an toàn.
- Unique Agent Identifiers (Định danh tác tử duy nhất): Phân quyền tài nguyên (RBAC) nghiêm ngặt cho từng tác tử với mã định danh độc lập.
- Human-in-the-loop (Giám sát con người có kiểm soát): Luôn yêu cầu sự phê duyệt trực tiếp của con người đối với các quyết định quan trọng (như phê duyệt giao dịch tài chính lớn hoặc kê đơn thuốc y tế nguy hiểm).

2: THỰC NGHIỆM CODE TRÊN NOTEBOOK

2.1 Môi Trường Tính Toán và Thiết Lập Ban Đầu

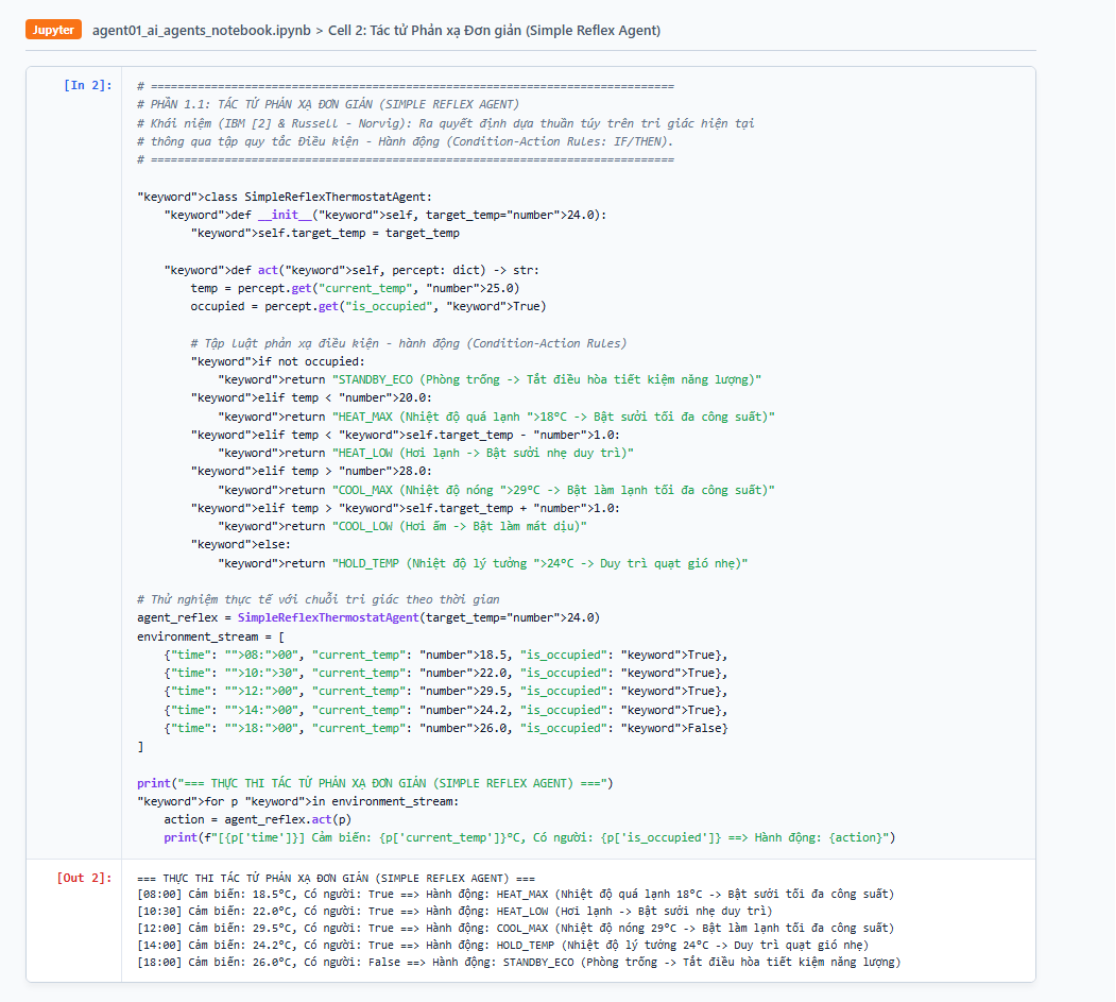
Cell 1 chịu trách nhiệm cấu hình môi trường Python, thiết lập các thư viện cốt lõi (random, numpy, json, collections, matplotlib) và hạt giống ngẫu nhiên để đảm bảo tính tái lập (Reproducibility) của thực nghiệm.



Hình 2.1: Ảnh chụp màn hình Cell 1 - Khởi tạo Môi trường và Thiết lập Ban đầu

2.2 Thực Nghiệm Tác Tử Phản Xạ Đơn Giản - Simple Reflex Agent

Cài đặt lớp SimpleReflexThermostatAgent: điều khiển hệ thống nhiệt độ thông minh dựa trên tập luật phản xạ IF/THEN tức thời (Current Temperature và Phòng có người hay không).



Hình 2.2: Ảnh chụp màn hình Cell 2 - Tác tử Phản xạ Đơn giản (Simple Reflex Agent)

2.3 Thực Nghiệm Tác Tử Dựa Trên Mô Hình Nội Tại - Model-based Agent

Cài đặt lớp ModelBasedVacuumAgent: tác tử robot hút bụi duy trì trạng thái nội tại (Internal State) lưu trữ bản đồ các phòng, tình trạng bụi bẩn và mức pin. Khi làm sạch xong một phòng, tác tử tự động tra cứu mô hình bộ nhớ để di chuyển sang phòng bẩn tiếp theo và tự động quay về trạm sạc khi pin dưới 15%.

Jupyter agent01_ai_agents_notebook.ipynb > Cell 3: Tác tử Dựa trên Mô hình Thế giới Nội tại (Model-based Reflex Agent)

[In 3]:

```
# ================================================================================
# PHẦN 1.2: TÁC TỬ DỰA TRÊN MÔ HÌNH (MODEL-BASED REFLEX AGENT)
# Khái niệm (IBM [2]): Duy trì trạng thái nội tại (Internal State) để theo dõi các khía cạnh
# của thế giới mà cảm biến hiện tại không nhìn thấy trực tiếp (Partial Observability).
# ================================================================================

class ModelBasedVacuumAgent:
    def __init__(self, rooms=["Phòng Khách", "Phòng Ngủ", "Bếp", "Phòng Làm Việc"]):
        self.rooms = rooms
        # Trạng thái nội tại (Internal State) lưu trữ bản đồ thế giới
        self.internal_state = {
            "current_location": "Phòng Khách",
            "room_dirt_status": {r: "DIRTY" for r in rooms},
            "cleaned_history": [],
            "battery_level": 100
        }

    def update_state(self, current_percept: dict):
        loc = current_percept["location"]
        dirt = current_percept["is_dirty"]
        self.internal_state["current_location"] = loc
        self.internal_state["room_dirt_status"][loc] = "DIRTY" if dirt else "CLEAN"
        self.internal_state["battery_level"] -= 5

    def choose_action(self) -> str:
        loc = self.internal_state["current_location"]
        status = self.internal_state["room_dirt_status"][loc]

        # Quyết định hành động dựa trên cả tri giác và mô hình nội tại
        if self.internal_state["battery_level"] <= 15:
            return "RETURN_TO_DOCK (Pin yếu -> Quay về trạm sạc)"

        if status == "DIRTY":
            self.internal_state["room_dirt_status"][loc] = "CLEAN"
            self.internal_state["cleaned_history"].append(loc)
            return f"SUCK_DIRT (Hút bụi dọn sạch tại {loc})"

        # Nếu phòng hiện tại đã sạch, tìm phòng bẩn tiếp theo trong mô hình bộ nhớ
        for r, s in self.internal_state["room_dirt_status"].items():
            if s == "DIRTY":
                return f"MOVE_TO (Di chuyển sang {r} còn bụi)"

        return "ALL_CLEAN (Tất cả các phòng trong bản đồ đã sạch -> Tự động nghỉ)"

# Mô phỏng quá trình dọn dẹp
agent_model = ModelBasedVacuumAgent()
percepts_log = [
    {"location": "Phòng Khách", "is_dirty": True},
    {"location": "Phòng Khách", "is_dirty": False},
    {"location": "Phòng Ngủ", "is_dirty": True},
    {"location": "Bếp", "is_dirty": True},
    {"location": "Phòng Làm Việc", "is_dirty": False}
]

print("=== THỰC THI TÁC TỬ DỰA TRÊN MÔ HÌNH (MODEL-BASED AGENT) ===")
for i, perc in enumerate(percepts_log, 1):
    agent_model.update_state(perc)
    act = agent_model.choose_action()
    print(f"Bước {i}: Vị trí = {perc['location']} | Bẩn = {perc['is_dirty']} | Pin = {agent_model.internal_state['battery_level']}%")
    print(f"  ↳ Hành động quyết định: {act}")
    print(f"  ↳ Bộ nhớ trạng thái nội tại: {agent_model.internal_state['room_dirt_status']}")
```

[Out 3]:

```
=== THỰC THI TÁC TỬ DỰA TRÊN MÔ HÌNH (MODEL-BASED AGENT) ===
Bước 1: Vị trí = Phòng Khách | Bẩn = True | Pin = 95%
  ↳ Hành động quyết định: SUCK_DIRT (Hút bụi dọn sạch tại Phòng Khách)
  ↳ Bộ nhớ trạng thái nội tại: {'Phòng Khách': 'CLEAN', 'Phòng Ngủ': 'DIRTY', 'Bếp': 'DIRTY', 'Phòng Làm Việc': 'DIRTY'}
Bước 2: Vị trí = Phòng Khách | Bẩn = False | Pin = 90%
  ↳ Hành động quyết định: MOVE_TO (Di chuyển sang Phòng Ngủ còn bụi)
  ↳ Bộ nhớ trạng thái nội tại: {'Phòng Khách': 'CLEAN', 'Phòng Ngủ': 'DIRTY', 'Bếp': 'DIRTY', 'Phòng Làm Việc': 'DIRTY'}
Bước 3: Vị trí = Phòng Ngủ | Bẩn = True | Pin = 85%
  ↳ Hành động quyết định: SUCK_DIRT (Hút bụi dọn sạch tại Phòng Ngủ)
  ↳ Bộ nhớ trạng thái nội tại: {'Phòng Khách': 'CLEAN', 'Phòng Ngủ': 'CLEAN', 'Bếp': 'DIRTY', 'Phòng Làm Việc': 'DIRTY'}
Bước 4: Vị trí = Bếp | Bẩn = True | Pin = 80%
  ↳ Hành động quyết định: SUCK_DIRT (Hút bụi dọn sạch tại Bếp)
  ↳ Bộ nhớ trạng thái nội tại: {'Phòng Khách': 'CLEAN', 'Phòng Ngủ': 'CLEAN', 'Bếp': 'CLEAN', 'Phòng Làm Việc': 'DIRTY'}
Bước 5: Vị trí = Phòng Làm Việc | Bẩn = False | Pin = 75%
  ↳ Hành động quyết định: ALL_CLEAN (Tất cả các phòng trong bản đồ đã sạch -> Tự động nghỉ)
  ↳ Bộ nhớ trạng thái nội tại: {'Phòng Khách': 'CLEAN', 'Phòng Ngủ': 'CLEAN', 'Bếp': 'CLEAN', 'Phòng Làm Việc': 'CLEAN'}
```

Hình 2.3: Ảnh chụp màn hình Cell 3 - Tác tử Dựa trên Mô hình Thế giới Nội tại

2.4 Thực Nghiệm Tác Tử Dựa Trên Mục Tiêu - Goal-based Pathfinder Agent

Cài đặt lớp GoalBasedPathfinderAgent: tác tử được giao mục tiêu Goal = (4, 4) xuất phát từ Start = (0, 0). Tác tử triển khai thuật toán tìm kiếm Breadth-First Search (BFS) để hoạch định chuỗi hành động tối ưu vượt qua các chướng ngại vật phức tạp trên bản đồ mê cung.



Hình 2.4: Ảnh chụp màn hình Cell 4 - Tác tử Dựa trên Mục tiêu (Goal-based Agent)

2.5 Thực Nghiệm Tác Tử Dựa Trên Hàm Thỏa Dụng - Utility-based Agent

Cài đặt lớp UtilityBasedDeliveryDroneAgent: tác tử máy bay không người lái giao hàng tự động. Đứng trước 3 tuyến đường bay ứng viên, tác tử tính toán hàm thỏa dụng đa mục tiêu U(s) = 0.4 * Tốc độ + 0.3 * Tiết kiệm pin + 0.3 * An toàn thời tiết để chọn ra tuyến đường tối ưu nhất.

Jupyter agent01_ai_agents_notebook.ipynb > Cell 5: Tác tử Dựa trên Hàm Thỏa dụng (Utility-based Agent)

[In 5]:

```
# ================================================================================
# PHẦN 1.4: TÁC TỬ DỰA TRÊN ĐỘ THỎA DỤNG (UTILITY-BASED AGENT)
# Khái niệm (IBM [2]): Khi có nhiều cách đạt mục tiêu, tác tử sử dụng Hàm thỏa dụng (Utility Function)
# để định lượng mức độ hài lòng, cân bằng tối ưu giữa Phần thưởng, Chi phí và Rủi ro.
# ================================================================================

class UtilityBasedDeliveryDroneAgent:
    def __init__(self, w_speed=0.4, w_cost=0.3, w_safety=0.3):
        # Trọng số ưu tiên của hàm thỏa dụng
        self.w_speed = w_speed
        self.w_cost = w_cost
        self.w_safety = w_safety

    def calculate_utility(self, route: dict) -> float:
        """
        Hàm Thỏa dụng: U(s) = w1 * Tốc độ - w2 * Chi phí pin + w3 * Độ an toàn thời tiết
        Chuẩn hóa các thành phần về thang điểm [0, 100].
        """
        speed_score = (60.0 / route["duration_mins"]) * 100.0
        cost_score = (1.0 - route["battery_cost_percent"] / 100.0) * 100.0
        safety_score = route["weather_safety_score"] # 0 - 100

        total_utility = (self.w_speed * speed_score +
                         self.w_cost * cost_score +
                         self.w_safety * safety_score)
        return round(total_utility, 2)

    def select_best_route(self, candidate_routes: list) -> tuple:
        best_route = None
        max_utility = -float('inf')

        for r in candidate_routes:
            u = self.calculate_utility(r)
            r["evaluated_utility"] = u
            if u > max_utility:
                max_utility = u
                best_route = r
        return best_route, candidate_routes

# 3 tuyến đường bay giao hàng ứng viên
routes = [
    {"name": "Tuyến A (Đường thẳng qua đồi)", "duration_mins": 15, "battery_cost_percent": 45, "weather_safety_score
    {"name": "Tuyến B (Đường vòng dọc bờ sông)", "duration_mins": 35, "battery_cost_percent": 25, "weather_safety_sc
    {"name": "Tuyến C (Đường trung gian)", "duration_mins": 22, "battery_cost_percent": 30, "weather_safety_score":
]

agent_drone = UtilityBasedDeliveryDroneAgent()
best_r, evaluated_list = agent_drone.select_best_route(routes)

print("=== THỰC THI TÁC TỬ DỰA TRÊN ĐỘ THỎA DỤNG (UTILITY-BASED AGENT) ===")
print("Đánh giá các tuyến đường bay dựa trên Hàm Thỏa Dụng U(s):")
for r in evaluated_list:
    print(f"• {r['name']}: Thời gian = {r['duration_mins']}p | Pin tốn = {r['battery_cost_percent']}% | An toàn = {r['weather_safety_s

print(f"\n==> QUYẾT ĐỊNH TỐI ƯU CỦA AGENT: Lựa chọn '{best_r['name']}' với điểm thỏa dụng cao nhất ({best_r['evaluated_utility']}).")
```

[Out 5]:

```
=== THỰC THI TÁC TỬ DỰA TRÊN ĐỘ THỎA DỤNG (UTILITY-BASED AGENT) ===
Đánh giá các tuyến đường bay dựa trên Hàm Thỏa Dụng U(s):
• Tuyến A (Đường thẳng qua đồi): Thời gian = 15p | Pin tốn = 45% | An toàn = 60/100 ==> Điểm Thỏa Dụng = 194.5
• Tuyến B (Đường vòng dọc bờ sông): Thời gian = 35p | Pin tốn = 25% | An toàn = 95/100 ==> Điểm Thỏa Dụng = 119.57
• Tuyến C (Đường trung gian): Thời gian = 22p | Pin tốn = 30% | An toàn = 85/100 ==> Điểm Thỏa Dụng = 155.59

==> QUYẾT ĐỊNH TỐI ƯU CỦA AGENT: Lựa chọn 'Tuyến A (Đường thẳng qua đồi)' với điểm thỏa dụng cao nhất (194.5).
```

Hình 2.5: Ảnh chụp màn hình Cell 5 - Tác tử Dựa trên Hàm Thỏa dụng (Utility-based Agent)

.

2.6 Thực Nghiệm Tác Tử Tự Học Thích Nghi Q-Learning - Learning

Cài đặt lớp QLearningGridAgent dựa trên tài liệu [4] GeeksforGeeks Reinforcement Learning: tác tử học tăng cường trong môi trường GridWorld 4x4. Thông qua việc tương tác và nhận phần thưởng (Reward = +100 khi tới đích, -50 khi rơi vào bẫy, -1 chi phí bước đi), tác tử cập nhật ma trận Q-Table bằng phương trình Bellman qua 250 tập huấn luyện (Episodes).



Hình 2.6: Ảnh chụp màn hình Cell 6 - Tác tử Học thích nghi Q-Learning (Reinforcement Learning [4])

2.7 Thực Nghiệm Mô Thức Suy Luận Hiện Đại ReAct Agent

Cài đặt lớp ReActAgent theo mô thức từ IBM [2]: tác tử tích hợp hai công cụ chuyên biệt là máy tính an toàn (calculator) và cơ sở dữ liệu biệt dược y tế (medical_db). Khi nhận câu hỏi phức tạp về chống chỉ định của thuốc Metformin và dự trù chi phí uống trong 60 ngày, tác tử thực thi chu trình tuần tự: Thought 1 -> Action 1 -> Observation 1 -> Thought 2 -> Action 2 -> Observation 2 -> Final Answer.

Jupyter agent01_ai_agents_notebook.ipynb > Cell 7: Mô thức Suy luận Hiện đại ReAct Agent (Thought - Action - Observation)

[In 7]:

```
# ============================================================================
# PHẦN 2: MÔ THỨC SUY LUẬN HIỆN ĐẠI REACT AGENT (REASONING AND ACTION)
# Khái niệm (IBM [2]): Kết hợp khả năng lập luận tư duy (Reasoning) và hành động (Action/Tool Calling)
# theo chu trình vòng lặp: Thought -> Action -> Observation -> Thought -> Final Answer.
# ============================================================================

# Khởi tạo Hệ thống Công cụ Ngoại vi (Tools Ecosystem)
def tool_calculator(expr: str) -> str:
    """Công cụ máy tính an toàn tính toán biểu thức số học."""
    try:
        clean_expr = expr.replace('^', '**').strip()
        res = eval(clean_expr, {"__builtins__": None}, {})
        return f"{res}"
    except Exception as e:
        return f"Lỗi tính toán: {e}"

def tool_medical_db(query_drug: str) -> str:
    """Công cụ tra cứu cơ sở tri thức biệt dược và chống chỉ định y tế."""
    db = {
        "metformin": "Thuốc đầu tay điều trị Đái tháo đường Tuýp 2. Chống chỉ định: Suy thận nặng (eGFR < 30 mL/phút). Đơn giá Lon
        "forxiga": "Thuốc ức chế SGLT-2, thải trừ đường qua thận và bảo vệ tim mạch. Đơn giá: 620.000 đ/hộp 28 viên.",
        "insulin": "Chỉ định Đái tháo đường Tuýp 1 hoặc Tuýp 2 thất bại thuốc uống. Bảo quản nhiệt độ 2-8°C."
    }
    key = query_drug.lower().strip()
    return db.get(key, "Không tìm thấy thông tin biệt dược trong cơ sở dữ liệu.")

class ReActAgent:
    def __init__(self):
        self.tools = {
            "calculator": tool_calculator,
            "medical_db": tool_medical_db
        }

    def run(self, user_prompt: str) -> str:
        trace = []
        trace.append(f"CÂU HỎI NGƯỜI DÙNG: {user_prompt}")

        # Bước 1: Suy luận (Thought 1)
        thought_1 = "Tôi cần tra cứu chống chỉ định của thuốc Metformin đối với bệnh nhân suy thận, sau đó tính chi phí dùng thuốc trc
        trace.append(f"Thought 1: {thought_1}")

        # Bước 2: Hành động (Action 1 - Gọi Tool medical_db)
        action_1 = "medical_db[metformin]"
        trace.append(f"Action 1: {action_1}")
        obs_1 = self.tools["medical_db"]("metformin")
        trace.append(f"Observation 1: {obs_1}")

        # Bước 3: Suy luận (Thought 2)
        thought_2 = "Cơ sở dữ liệu nêu rõ Metformin chống chỉ định với suy thận nặng (eGFR < 30). Về chi phí: 1 hộp 50 viên giá
        trace.append(f"Thought 2: {thought_2}")

        # Bước 4: Hành động (Action 2 - Gọi Tool calculator)
        action_2 = "calculator[120 * 2700]"
        trace.append(f"Action 2: {action_2}")
        obs_2 = self.tools["calculator"]("120 * 2700")
        trace.append(f"Observation 2: {obs_2} (VNĐ)")

        # Bước 5: Tổng hợp câu trả lời cuối cùng (Final Answer)
        final_answer = (
            f"1. Cảnh báo an toàn: Bệnh nhân suy thận nặng (eGFR < 30) CHỐNG CHỈ ĐỊNH tuyệt đối dùng Metformin do nguy cơ nhiễm tc
            f"2. Dự trù chi phí (nếu đủ điều kiện dùng 2 viên/ngày): Hết 120 viên trong 60 ngày, tổng chi phí là {int(float(ob
        )
        trace.append(f"Final Answer:\n{final_answer}")
        return "\n".join(trace)

agent_react = ReActAgent()
output_trace = agent_react.run("Bệnh nhân suy thận có dùng được Metformin không và nếu uống 2 viên/ngày trong 60 ngày thì hết bao

print("=== THỰC THI MÔ THỨC SUY LUẬN REACT AGENT (REASONING + ACTION) ===")
print(output_trace)
```

[Out 7]:

```
=== THỰC THI MÔ THỨC SUY LUẬN REACT AGENT (REASONING + ACTION) ===
CÂU HỎI NGƯỜI DÙNG: Bệnh nhân suy thận có dùng được Metformin không và nếu uống 2 viên/ngày trong 60 ngày thì hết bao nhiêu tiền?
Thought 1: Tôi cần tra cứu chống chỉ định của thuốc Metformin đối với bệnh nhân suy thận, sau đó tính chi phí dùng thuốc trong 60 ngày.
Action 1: medical_db[metformin]
Observation 1: Thuốc đầu tay điều trị Đái tháo đường Tuýp 2. Chống chỉ định: Suy thận nặng (eGFR < 30 mL/phút). Đơn giá Long Châu:
135.000 đ/hộp 50 viên.
Thought 2: Cơ sở dữ liệu nêu rõ Metformin chống chỉ định với suy thận nặng (eGFR < 30). Về chi phí: 1 hộp 50 viên giá 135.000đ
(2.700đ/viên). Dùng 60 ngày x 2 viên/ngày = 120 viên. Tôi cần tính tổng tiền 120 * 2700.
Action 2: calculator[120 * 2700]
Observation 2: 324000 (VNĐ)
Final Answer:
1. Cảnh báo an toàn: Bệnh nhân suy thận nặng (eGFR < 30) CHỐNG CHỈ ĐỊNH tuyệt đối dùng Metformin do nguy cơ nhiễm toan lactic. Cần hỏi
bác sĩ để chuyển sang nhóm thuốc an toàn cho thận (như DPP-4 Linagliptin).
2. Dự trù chi phí (nếu đủ điều kiện dùng 2 viên/ngày): Hết 120 viên trong 60 ngày, tổng chi phí là 324,000 VNĐ tại Nhà thuốc Long Châu.
```

Hình 2.7: Ảnh chụp màn hình Cell 7 - Mô thức Suy luận Hiện đại ReAct Agent (Reasoning + Action)

2.8 Thực Nghiệm Hệ Đa Tác Tử Cộng Tác - Multi-Agent Collaborative System

Cài đặt hệ thống gồm 3 tác tử phối hợp nhịp nhàng: PlannerAgent (Phân rã nhiệm vụ chiến lược thành các bước con) -> WorkerAgent (Thực thi từng công việc chuyên môn và tra cứu số liệu) -> CriticAgent (Thẩm định an toàn lâm sàng và phê duyệt phương án).



Hình 2.8: Ảnh chụp màn hình Cell 8 - Hệ Đa Tác tử Cộng tác (Multi-Agent System: Planner - Worker - Critic)

TÀI LIỆU THAM KHẢO

[1] Google Cloud, 'What are AI agents? Definition, examples, and types,' Google Cloud Discover, 2024. [Online]. Available: https://cloud.google.com/discover/what-are-ai-agents

[2] A. Gutowska, 'What Are AI Agents? Architecture, Paradigms and Best Practices,' IBM Think Topics, July 2024. [Online]. Available: https://www.ibm.com/think/topics/ai-agents

[3] McKinsey & Company, 'What is an AI agent and how will they impact the world?,' McKinsey Explainers (QuantumBlack, AI by McKinsey), 2024. [Online]. Available: https://www.mckinsey.com/featured-insights/mckinsey-explainers/what-is-an-ai-agent

[4] GeeksforGeeks, 'What is Reinforcement Learning? - An Introduction to RL Algorithms and Concepts,' GeeksforGeeks Machine Learning, 2024. [Online]. Available: https://www.geeksforgeeks.org/machine-learning/what-is-reinforcement-learning/

[5] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Hoboken, NJ: Pearson, 2020.

[6] S. Yao et al., 'ReAct: Synergizing reasoning and acting in language models,' International Conference on Learning Representations (ICLR), 2023.

[7] B. Xu et al., 'ReWOO: Decoupling reasoning from observations for efficient augmented language models,' arXiv preprint arXiv:2305.18323, 2023.

[8] L. Wang et al., 'A survey on large language model based autonomous agents,' Frontiers of Computer Science, vol. 18, no. 6, pp. 1-26, 2024.

[9] R. Sutton and A. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA: MIT Press, 2018.

[10] Trần Đình Quế, 'Phát triển Hệ thống Thông minh: Bài giảng chuyên đề Deep Learning, Knowledge Graph, RAG và Multi-Agent Systems,' Học viện Công nghệ Bưu chính Viễn thông, 2026.